

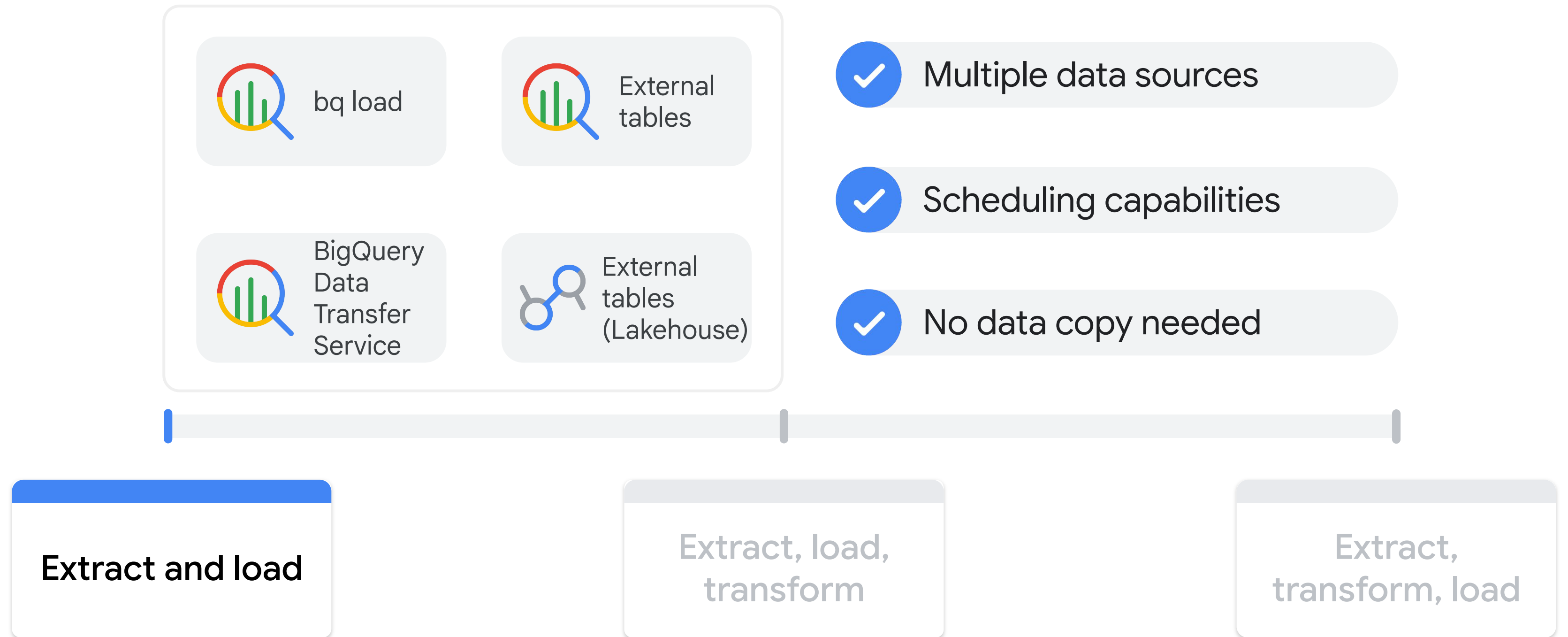
M3 — What this block gets you

- ✓ Extract & Load = your Bronze-landing pattern for files— land the file in BigQuery as-is, transform later
- ✓ One load job, three front doors— console form, bq load, and LOAD DATA SQL; batch loads are all free
- ✓ Autodetect is a first look, not a pipeline— pinned schemas in source control for anything recurring
- ✓ Data Transfer Service = scheduled ingest— the managed answer to your SQL Agent import jobs
- ✓ BigLake / external tables = query files where they live— the bridge to Day 2's lakehouse design

After lunch: land the files, 13:15–14:45

- 13:15–13:45** M3 Extract & Load — EL as Bronze landing; bq CLI; Data Transfer Service; BigLake / external tables
- 13:45–14:30** Loading block, four free DINs — console & CLI load → autodetect pitfall → CSV vs Parquet race → LOAD DATA SQL **DIN**
- 14:30–14:45** Break

Extract and load ingests data into BigQuery without the need for transformation



BigQuery supports importing from and exporting to multiple formats

Loading data

- Avro
- Parquet
- ORC
- CSV
- JSON
- Firestore export



Query results

- CSV
- JSON
- Google Sheets

Table export

- CSV
- JSON
- Avro
- Parquet

Load data into BigQuery by using the UI or the **LOAD DATA** statement

Create table from

Upload ▼

Select file **BROWSE**

File format

CSV ▼

Schema

☒ Auto detect

```
# create a new table by loading data from  
# CSV files using schema auto-detect  
LOAD DATA INTO dataset_name.table_name  
FROM FILES(  
  format='CSV',  
  uris = ['gs://mybucket/*.csv']  
)
```



Use **LOAD DATA INTO** to append to an existing table.

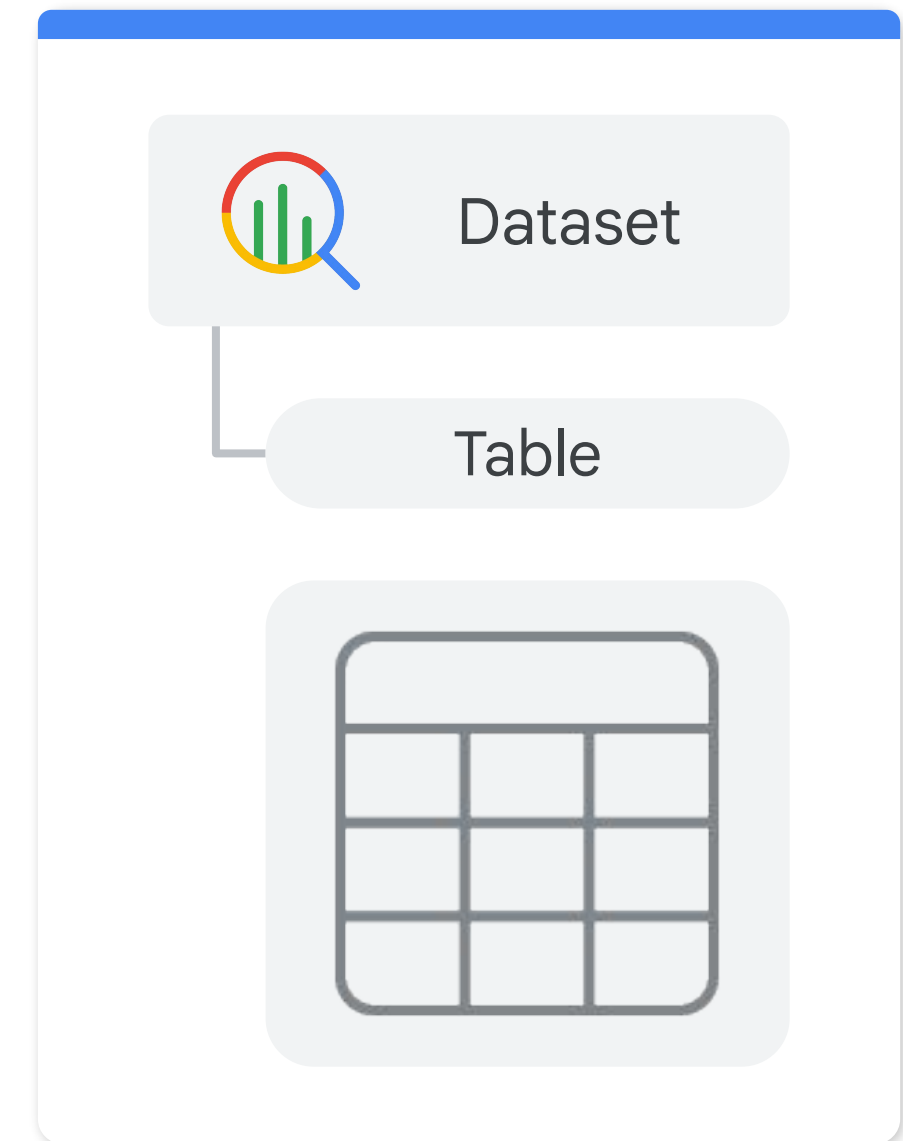
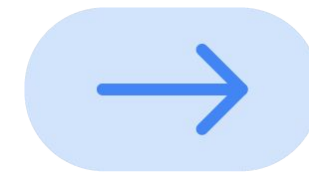


Use **LOAD DATA OVERWRITE** to overwrite the existing table.

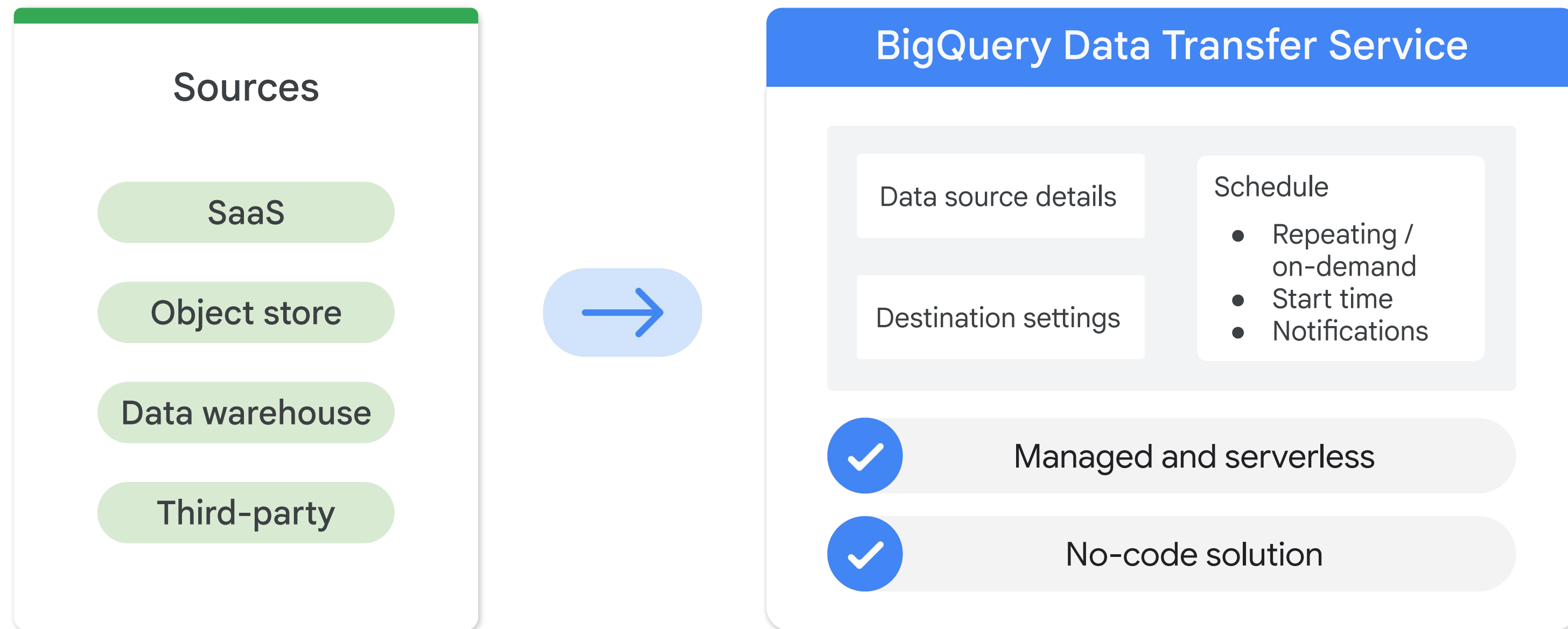
Interact with BigQuery by using the `bq` command

```
> bq mk --location=US -dataset dataset-name
```

```
> bq load \  
  --source_format=CSV \  
  --skip_leading_rows=2 \  
  dataset-name.table_name \  
  "gs://mybucket/00/*.csv", "gs://mybucket/01/*.csv" \  
  ./table_schema.json
```

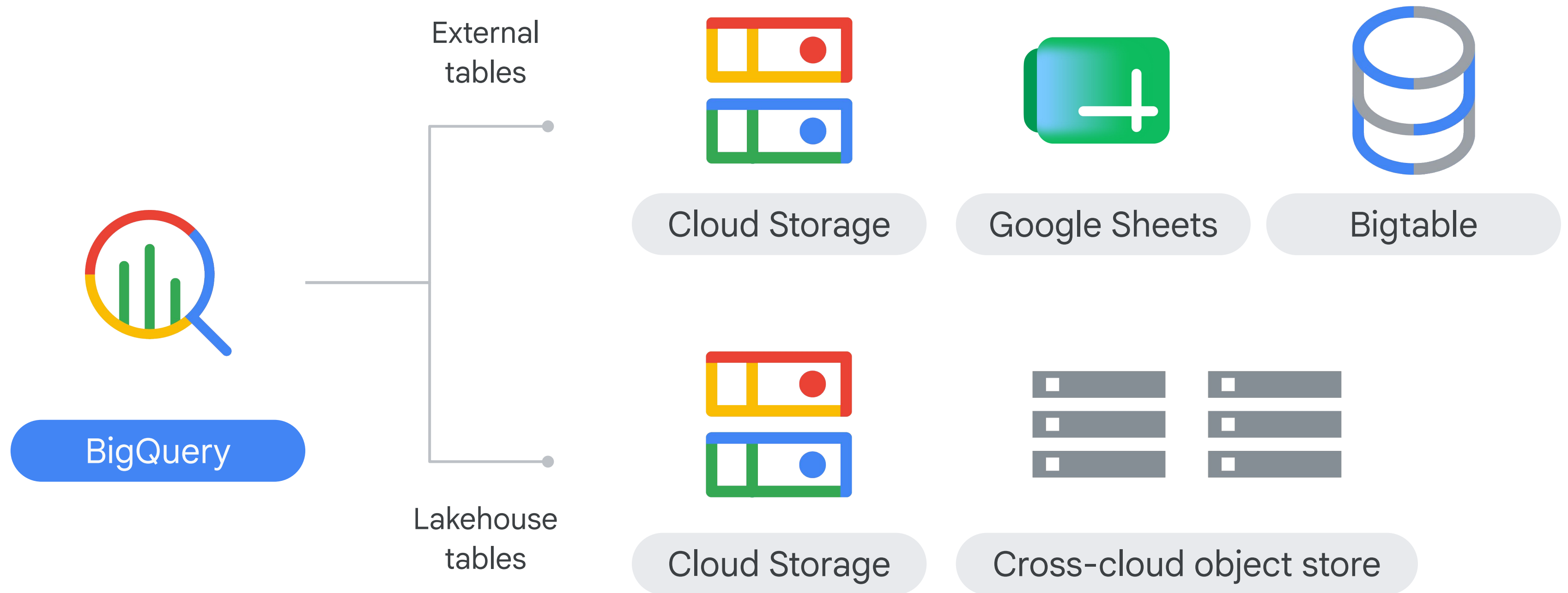


Use the BigQuery Data Transfer Service to load data from other structured data sources

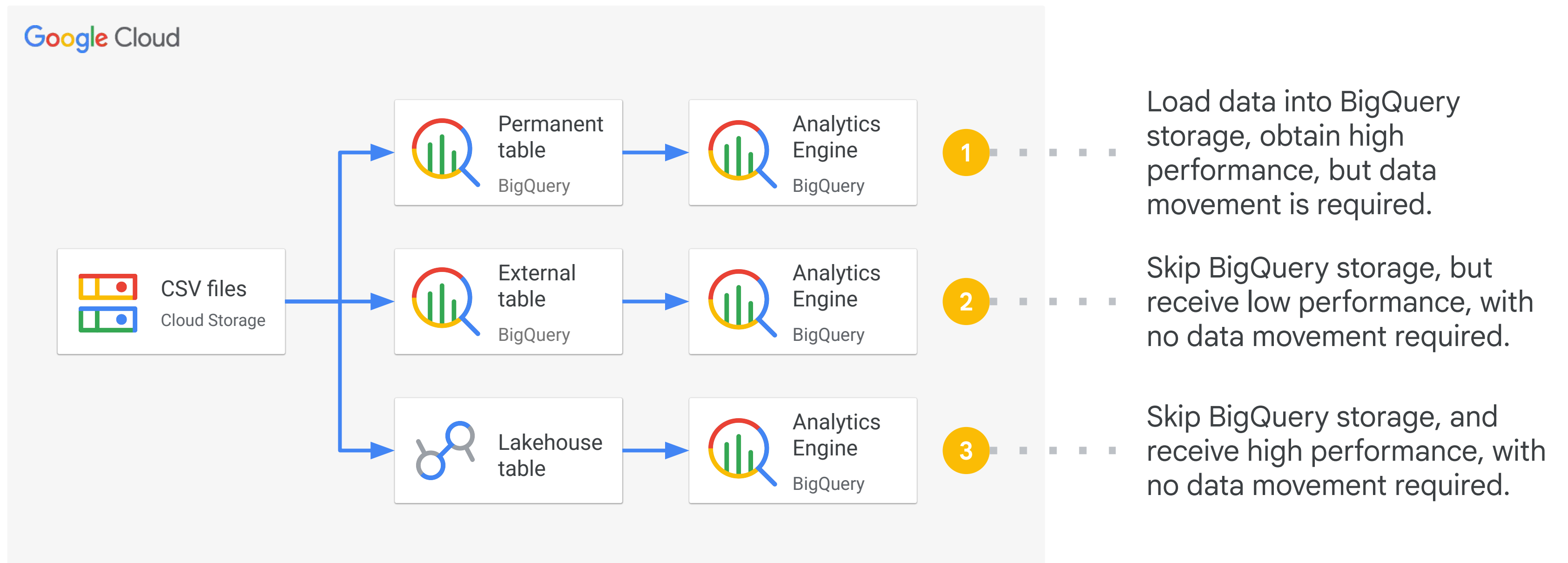


Your data isn't stored in BigQuery?

You can still query it!



Three ways of analyzing structured data in BigQuery



External tables let you query Google Sheets data directly in BigQuery

SCHEMADETAILSLINEAGE...

Table information

Table ID	your-project-id.sheets.sheets_table
...	

External data configuration

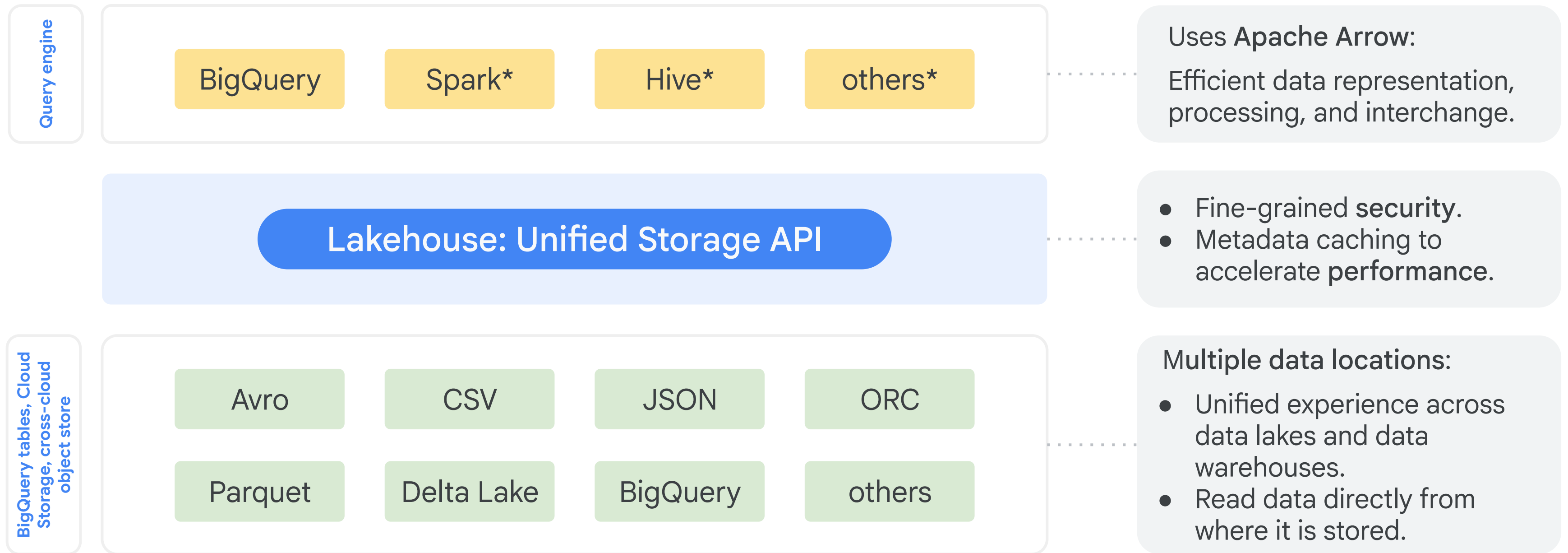
Source URI(s)	https://docs.google.com/spreadsheets/...
Source format	GOOGLE_SHEETS



```
# Query the Google Sheets table
SELECT ...
FROM
  `your-project-id.sheets.sheets_table`;
```

-  No query cost estimation, table preview, or query caching available.
-  Performance is slower than querying a permanent table in BigQuery.

Lakehouse: BigQuery on your data lake... and much more!



A Lakehouse table behaves the same way as a permanent table

SCHEMADETAILSLINEAGE...

Table information

Table ID	your-project-id.biglake.csv_table
...	

External data configuration

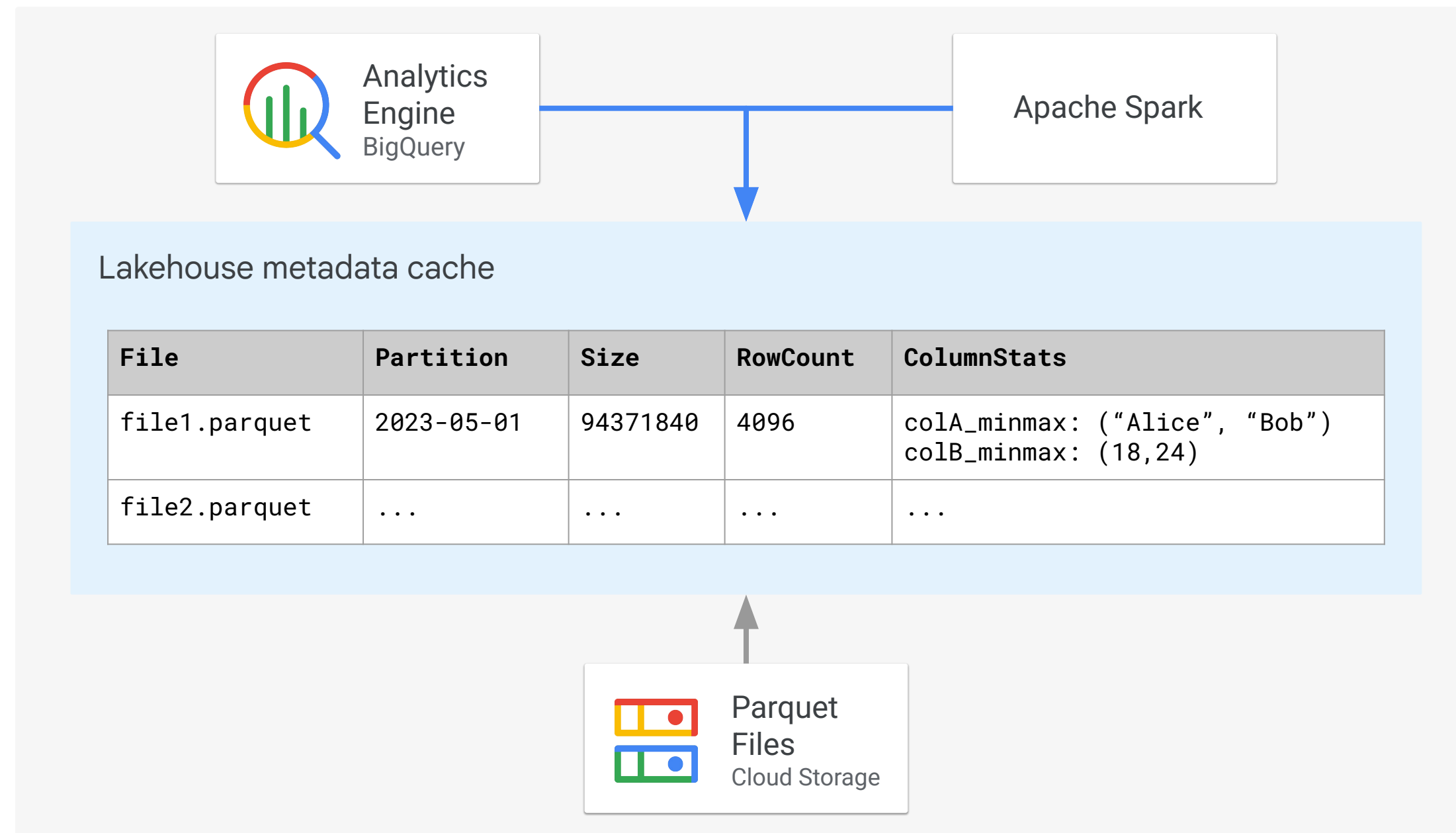
Source URI(s)	gs://your-bucket/your_file.csv
...	



```
# Query the Lakehouse table
SELECT ...
FROM
  `your-project-id.biglake.csv_table`;
```

- ✓ Use Lakehouse if loading data into BigQuery isn't an option for your use case.
- ✓ Queries are performant due to metadata caching.
- ✗ No query cost estimation or table preview.

Lakehouse uses metadata caching to accelerate performance



Advantages of BigQuery

- Skip object list
- Faster file and partition pruning
- Dynamic predicate pruning

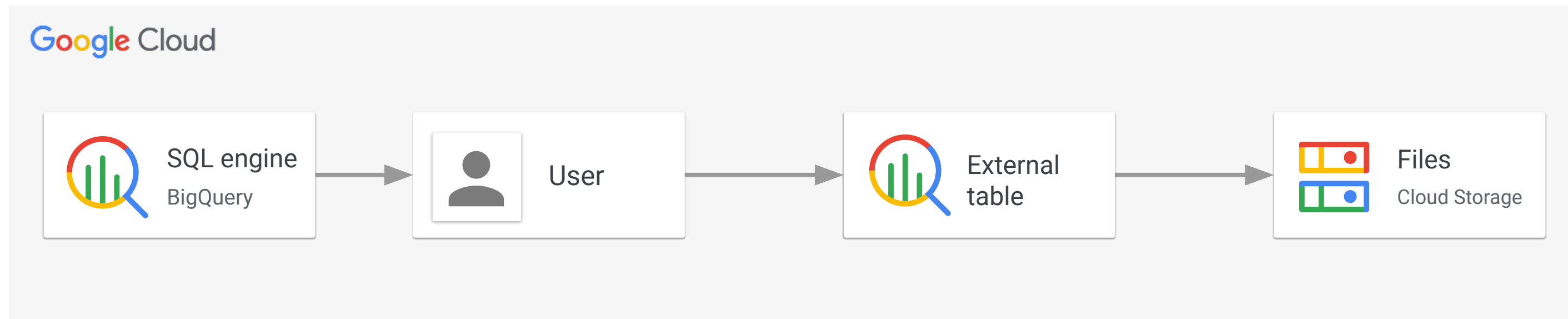
Advantages of Spark

- Can read metadata statistics using the spark-bigquery connector.

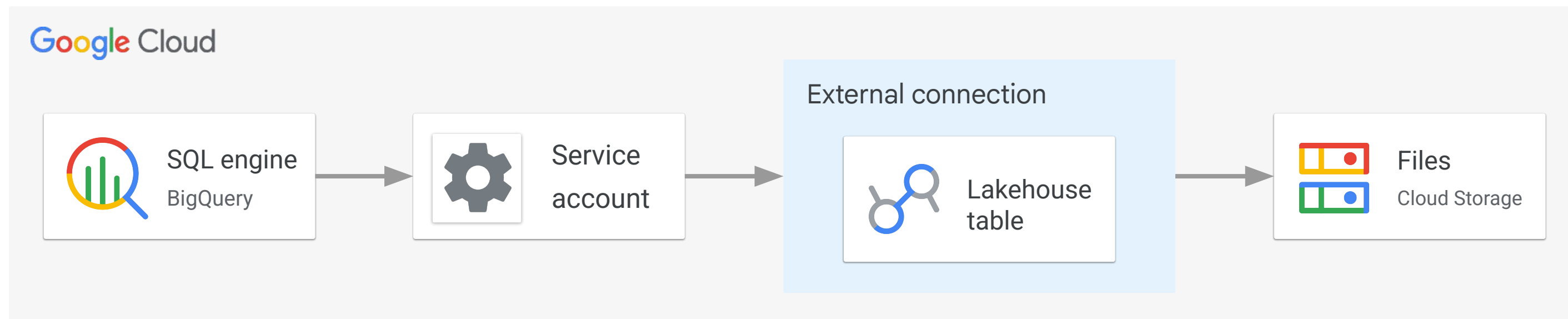
Cache properties

- Staleness between 30 min and 7 days
- Automatic or manual refresh

Compare security approaches: Non-native stored data

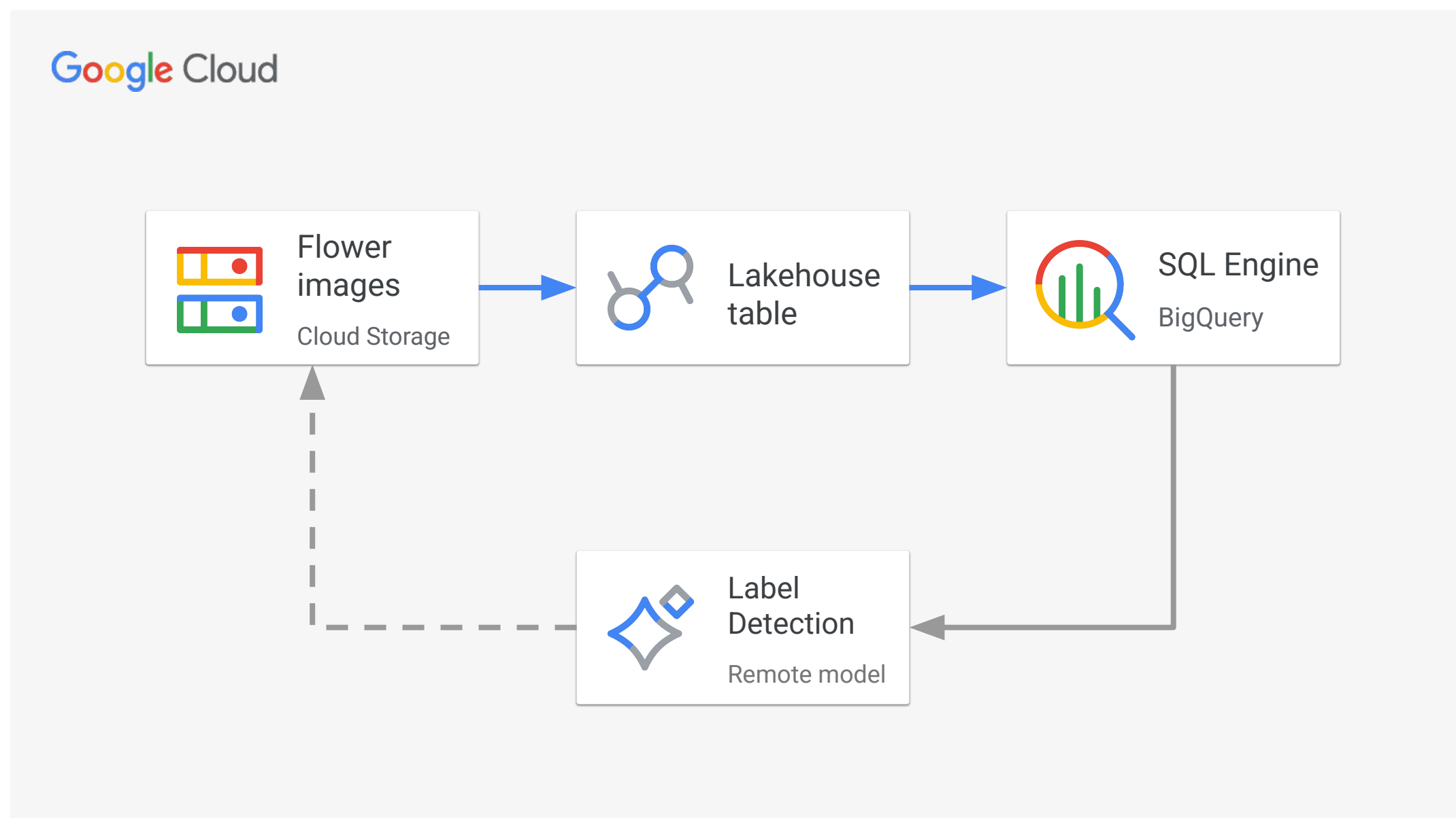


External tables: the user needs separate permissions to access the table and the data source.



Lakehouse: access is delegated using a service account. It decouples access to the table from the underlying data source.

[Demo] Analyze an object table by using a remote model



```
gs://biglake-flowers/flower_photos/  
daisy/pic.jpg
```

```
"label_annotations": [{  
  "description": "Flower",  
  "score": 0.98062  
}]
```

Compare external and Lakehouse tables

	External tables	Lakehouse tables
Query Engine	BigQuery	BigQuery, others using BigQuery connector
Data types	- Parquet, Avro, JSON, ORC, CSV - Datastore export, Firestore export - Google Sheets	- Parquet, Avro, JSON, ORC, CSV - Apache Iceberg, Delta, Hudi - Hive partitioned - Any unstructured (via object tables)
Locations	- Cloud Storage - Bigtable - Google Drive	- Cloud Storage - Amazon S3 - Azure Blob Storage
Access pattern	Direct link to the source. The user needs to have access to the source	External connection with access delegation. The service account has access to the source.
Security	None	- Column-level security, data masking - Row-level security

DIN: Loading data — console and CLI

~10 minutes · BigQuery console + Cloud Shell — into your class dataset

1. Console: Create table on your class dataset from `jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv`
2. Details tab: 1,000,000 rows; autodetect typed minutes as INTEGER, event_date as DATE
3. Append Tuesday's file: `bq load --autodetect --source_format=CSV class.usage gs://.../usage_2026-07-21.csv`
4. Refresh: 2,000,000 rows — loads append by default; `--replace` truncates first
5. View all jobs: console and bq load are two front ends for the same free load job

Why it matters — your Bronze-landing pattern in miniature — vendor file → bucket → BigQuery table, the same shape as the extract jobs you are replacing.

DIN: The autodetect pitfall — N/A in a numeric column

~10 minutes · Cloud Shell + BigQuery console

1. Paste `autodetect_demo.sh` into Cloud Shell — two vendor drops with N/A in the numeric minutes column
2. Load Monday (100k rows, N/A at row 90,000) with `--autodetect`: fails loudly — Unable to parse
3. Load Tuesday (1,000 rows, N/A at row 700) the same way: succeeds, but minutes lands as `STRING`
4. `SUM(minutes)` fails — loud failure vs silent wrong schema, decided by the sample window
5. Fix: pinned schema JSON + `--null_marker='N/A'` — 101,000 rows, 2 NULLs, `SUM` works

Why it matters — autodetect is the SSIS "Suggest Types" button — fine for a first look, never for a recurring feed. The silent failure is the one that costs you days.

DIN: CSV vs Parquet race — 60 million rows, two ways

~10 minutes · Cloud Shell — `gs://jwd-gcp-demos/ingest_demo`

1. Same 60M rows staged two ways: one 317 MiB gzipped CSV vs 100 Parquet files (~829 MiB total)
2. Load the gzip: `bq load --source_format=CSV` with an inline schema — watch the counter, about two minutes
3. While it runs: a gzip cannot be split — one worker decompresses and parses the whole file alone
4. Load the Parquet copy: `bq load --source_format=PARQUET` — done in seconds; Parquet carries its own schema
5. `bq show` both tables: 60,000,000 rows each — more than twice the bytes, roughly 20x faster

Why it matters — your Bronze-layer format decision: land Parquet wherever you control the format; for vendor CSVs you don't, prefer many small files over one large gzip.

DIN: LOAD DATA — the load job goes SQL

~5–8 minutes · BigQuery query editor — `load_data_example.sql`

1. Run the load-monday query: `LOAD DATA INTO class.usage_sql` with a column list pinning the schema, `FROM FILES CSV` options
2. Run the load-tuesday query: no column list — the table exists; `LOAD DATA INTO` appends, `OVERWRITE` would replace
3. Run the verify query: each date holds 1,000,000 meetings and `SUM(minutes)` works
4. Check Job information on either load: Bytes billed = 0 B — SQL syntax, same free load machinery

Why it matters — the load is now a statement that lives in a script, procedure, or scheduled query — one scheduled `LOAD DATA` with a date-built URI replaces a nightly extract job.

M3 — What you now have

- ✓ The EL pattern mapped: console, bq load, and LOAD DATA all submit the same free load job into BigQuery
- ✓ Four loads with your own hands— console & CLI, the autodetect N/A trap, the 60M-row format race, LOAD DATA SQL
- ✓ Formats and pinned schemas are Bronze-layer design decisions, not conveniences
- ✓ Scheduled ingest covered two ways: Data Transfer Service, or a scheduled query running LOAD DATA
- ✓ Next: break, then M4 ELT— transform where the data already lives, and your first look at Dataform